Лабораторная работа №2. Первоначальная настройка git

Дисциплина: Архитектура компьютеров и операционные системы

Мацюк Константин Владимирович

Содержание

1 Цель работы **5**

2 Задание **6**

3 Выполнение лабораторной работы **7**

3.1 Установка программного обеспечения 7
3.2 Базовая настройка git 7
3.3 Создание ключей SSH 8
3.4 Добавление SSH ключа на GitHub 8
3.5 Создание ключа PGP 9
3.6 Добавление PGP ключа на GitHub 10
3.7 Настройка автоматических подписей коммитов 10
3.8 Настройка gh (GitHub CLI) 10
3.9 Создание репозитория курса 11

4 Ответы на контрольные вопросы **12**

5 Выводы **16**

Список литературы **17**

Список иллюстраций

3.1 Установка git и gh 7
3.2 Базовая настройка git 7
3.3 Генерация RSA ключа 8
3.4 Добавление SSH ключа на GitHub 8
3.5 Генерация PGP ключа 9
3.6 Просмотр списка PGP ключей 9
3.7 Экспорт PGP ключа 10
3.8 Добавление GPG ключа на GitHub 10
3.9 Настройка автоподписи коммитов 10
3.10 Авторизация gh и проверка статуса 11
3.11 Создание репозитория на основе шаблона 11
3.12 Настройка каталога курса 11

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

2 Задание

- Создать базовую конфигурацию для работы с git
- Создать ключ SSH
- Создать ключ PGP
- Настроить подписи git
- Зарегистрироваться на Github
- Создать локальный каталог для выполнения заданий по предмету

3 Выполнение лабораторной работы

3.1 Установка программного обеспечения

Устанавливаю git и gh с помощью dnf (рис. 3.1).

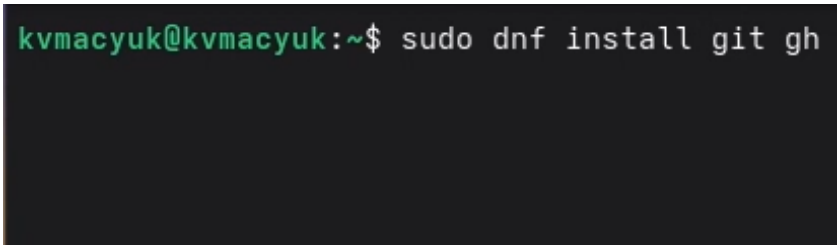

Рис. 3.1: Установка git и gh

3.2 Базовая настройка git

Выполняю базовую настройку git: задаю имя и email владельца репозитория, настраиваю utf-8 в выводе сообщений, задаю имя начальной ветки master, настраиваю параметры autocrlf и safecrlf (рис. 3.2).

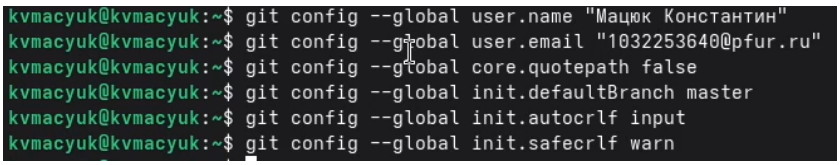

Рис. 3.2: Базовая настройка git

3.3 Создание ключей SSH

Создаю SSH ключ по алгоритму RSA с размером 4096 бит (рис. 3.3).

```
kvmacyuk@kvmacyuk:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kvmacyuk/.ssh/id_rsa):
Created directory '/home/kvmacyuk/.ssh'.
Enter passphrase for "/home/kvmacyuk/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kvmacyuk/.ssh/id_rsa
Your public key has been saved in /home/kvmacyuk/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:MdXMrmN0uTzgX4gurMkk8pX56OtznS1vjkgB5Qi4T9E kvmacyuk@kvmacyuk
The key's randomart image is:
+---[RSA 4096]----+
|   ...  . .+     |
|  . ..E+ .  +    |
|   . .o +  . .   |
|  . .  . oo +    |
|   o    So * o   |
|    .  o .* = .  |
|  . . =..+ = o   |
|   o =.=+.=.+    |
|    .oO=o..=o    |
+----[SHA256]-----+
```

Рис. 3.3: Генерация RSA ключа

3.4 Добавление SSH ключа на GitHub

Перехожу в настройки GitHub, добавляю ключ с названием Lab (рис. 3.4).

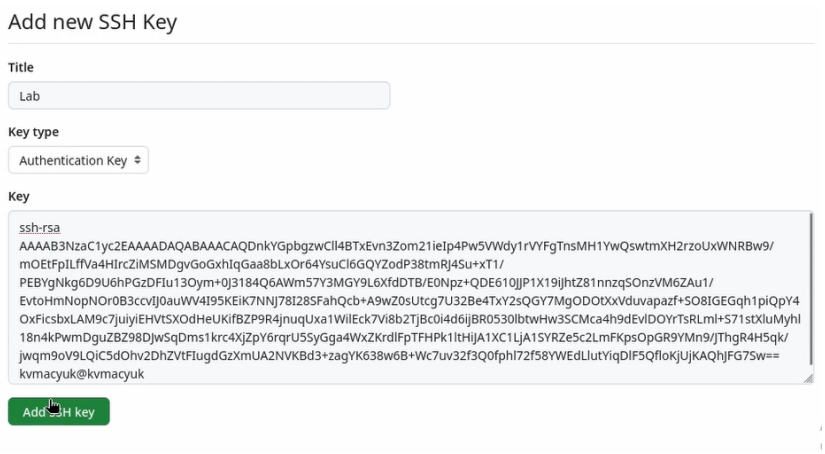

Рис. 3.4: Добавление SSH ключа на GitHub

3.5 Создание ключа PGP

Генерирую PGP ключ командой gpg –full-generate-key. Выбираю тип RSA and RSA, размер 4096 бит, срок действия 0. Ввожу имя и email (рис. 3.5).

```
kvmacyuk@kvmacyuk:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/kvmacyuk/.gnupg'
Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
```

Рис. 3.5: Генерация PGP ключа

Просматриваю список ключей и копирую fingerprint ключа (рис. 3.6).

```
Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? 0
Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход?
Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? O
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: /home/kvmacyuk/.gnupg/trustdb.gpg: создана таблица доверия
gpg: создан каталог '/home/kvmacyuk/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/home/kvmacyuk/.gnupg/openpgp-revocs.d/8228E87
66713A25BA795E92230C46763CA3FCCC8.rev'.
открытый и секретный ключи созданы и подписаны.

pub   rsa4096 2026-03-06 [SC]
      8228E8766713A25BA795E92230C46763CA3FCCC8
uid                      Константин <1032253640@rudn.ru>
sub   rsa4096 2026-03-06 [E]
```

Рис. 3.6: Просмотр списка PGP ключей

Экспортирую ключ для добавления на GitHub и копирую в буфер обмена (рис. 3.7).

```
kvmacyuk@kvmacyuk:~$ gpg --armor --export 30C46763CA3FCCC8 | xclip -sel clip
```

Рис. 3.7: Экспорт PGP ключа

3.6 Добавление PGP ключа на GitHub

Перехожу в настройки GitHub, вставляю экспортированный ключ (рис. 3.8).

Рис. 3.8: Добавление GPG ключа на GitHub

3.7 Настройка автоматических подписей коммитов

Настраиваю git для автоматической подписи коммитов с использованием созданного PGP ключа (рис. 3.9).

```
kvmacyuk@kvmacyuk:~$ git config --global user.signingkey 30C46763CA3FCCC8
kvmacyuk@kvmacyuk:~$ git config --global commit.gpgsign true
kvmacyuk@kvmacyuk:~$ git config --global gpg.program $(which gpg2)
```

Рис. 3.9: Настройка автоподписи коммитов

3.8 Настройка gh (GitHub CLI)

Авторизуюсь в GitHub CLI (рис. 3.10).

```
kvmacyuk@kvmacyuk:~$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/kvmacyuk/.ssh/id_rsa.
pub
? Title for your SSH key: Lab
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 0DA1-2C8D
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
✓ SSH key already existed on your GitHub account: /home/kvmacyuk/.ssh/id_rsa.pub
✓ Logged in as kvmacyuk
```

Рис. 3.10: Авторизация gh и проверка статуса

3.9 Создание репозитория курса

Создаю каталог для хранения репозитория курса и клонирую шаблон (рис. 3.11).

```
kvmacyuk@kvmacyuk:~$ mkdir -p ~/work/study/2025-2026/"Операционные системы"
kvmacyuk@kvmacyuk:~$ cd ~/work/study/2025-2026/"Операционные системы"
kvmacyuk@kvmacyuk:~/work/study/2025-2026/Операционные системы$ gh repo create st
udy_2025-2026_os-intro --template=yamadharma/course-directory-student-template -
-public
✓ Created repository kvmacyuk/study_2025-2026_os-intro on github.com
  https://github.com/kvmacyuk/study_2025-2026_os-intro
kvmacyuk@kvmacyuk:~/work/study/2025-2026/Операционные системы$ git clone --recur
sive git@github.com:kvmacyuk/study_2025-2026_os-intro.git os-intro
Клонирование в «os-intro»...
```

Рис. 3.11: Создание репозитория на основе шаблона

Настраиваю каталог курса: удаляю лишние файлы, создаю файл COURSE и запускаю make (рис. 3.12).

```
kvmacyuk@kvmacyuk:~/work/study/2025-2026/Операционные системы/os-intro$ rm packa
ge.json
kvmacyuk@kvmacyuk:~/work/study/2025-2026/Операционные системы/os-intro$ echo os-
intro > COURSE
```

Рис. 3.12: Настройка каталога курса

Репозиторий успешно создан.

4 Ответы на контрольные вопросы

1. Что такое системы контроля версий (VCS)?

VCS — это программное обеспечение, предназначенное для регистрации изменений, вносимых в файлы проекта. Оно даёт возможность сохранять состояния файлов, восстанавливать более ранние версии, а также организовать совместную работу нескольких специалистов над общей кодовой базой, минимизируя конфликты.

2. Хранилище, коммит, история, рабочая копия:

- **Хранилище (репозиторий)** - хранилище всех версии файлов и журнал всех изменений проекта.
- **Коммит** - это зафиксированное состояние файлов проекта на конкретный момент, сопровождаемое комментарием о проделанных изменениях.
- **История** — это хронологический список всех коммитов, содержащий информацию об авторе, времени и комментарии к каждому изменению.
- **Рабочая копия** — это актуальное содержимое файлов проекта, доступное разработчику на его локальном компьютере для редактирования.

3. Централизованные и распределённые VCS:

- **Централизованные** — предполагают наличие единственного сервера с полной историей версий, а разработчики получают только рабочие копии с этого сервера. К таким системам относятся CVS и Subversion (SVN).
- **Распределённые** — предоставляют каждому участнику полную локальную копию репозитория, включая всю историю. Примерами являются Git и Mercurial.

4. Единоличная работа с хранилищем

- Инициализация локального хранилища
- Добавление файлов под контроль версий
- Фиксация изменений с описанием
- Просмотр истории изменений
- Возврат к предыдущим версиям при необходимости

5. Работа с общим хранилищем

- Получение актуальной версии проекта из общего хранилища
- Внесение изменений в локальной рабочей копии
- Фиксация изменений в локальном репозитории
- Отправка зафиксированных изменений в общее хранилище
- Разрешение конфликтов при одновременном изменении одних файлов

6. Основные функции Git:

Ключевые возможности Git включают: контроль изменений в файлах, ведение подробной истории версий, создание и управление ветками, объединение изменений от нескольких участников и возможность отката к предыдущим состояниям проекта.

7. Команды Git:

Основные команды Git:

Команда	Описание
`git init`	Создание нового репозитория
`git clone`	Клонирование удалённого репозитория
`git add`	Добавление файлов в индекс
`git commit`	Фиксация изменений
`git push`	Отправка изменений на сервер
`git pull`	Получение изменений с сервера

Команда	Описание
`git status`	Просмотр состояния рабочей копии
`git log`	Просмотр истории коммитов
`git branch`	Управление ветками
`git merge`	Слияние веток

8. Примеры действий с репозиторием:

Локальный репозиторий:

```
git init
echo "hello" > file.txt
git add file.txt
git commit -m "начальный коммит"
```

Удалённый репозиторий:

```
git remote add origin git@github.com:user/repo.git  # привязка удалённого репозитория
git push -u origin master  # отправка коммитов в основную ветку удалённого репозитория
git pull  # получение актуальных данных с сервера
```

9. Назначение веток (branches):

Ветвление позволяет вести разработку новых возможностей или исправление ошибок изолированно от стабильной основной версии кода. По завершении работы изменения из созданной ветки интегрируются в основную. Такой подход предотвращает попадание неготового кода в рабочую версию продукта.

10. Исключение файлов из коммитов:

Для автоматического игнорирования определённых файлов при коммитах используется файл `.gitignore`, размещаемый в корневой папке репозитория. В нём задаются шаблоны имён для исключения:

```
*.log
```

```
*.tmp
node_modules/
build/
```

Это необходимо, чтобы временные файлы, результаты компиляции, конфиденциальная информация и другие служебные данные не попадали в систему контроля версий.

5 Выводы

В ходе выполнения лабораторной работы изучил идеологию и применение средств контроля версий. Освоил базовую настройку git, создание SSH и PGP ключей, настройку подписей коммитов. Настроил GitHub CLI (gh) для удобной работы с GitHub из командной строки. Создал репозиторий курса на основе шаблона и настроил его для дальнейшей работы.

Список литературы